Actividad Python - 20 Ejercicios

Guía de ejecución y funcionamiento

Cómo ejecutar los ejercicios

1. Instala Python 3. En Windows, durante la instalación conviene activar la opción para agregar Python al PATH.
2. Abre una terminal ubicada dentro de la carpeta de la actividad.
3. Comprueba la instalación con python --version. En algunos equipos Windows también puedes usar py --version.
4. Ejecuta un ejercicio, por ejemplo: python 11_calculadora_imc.py.

También puedes abrir cualquier archivo en VS Code, IDLE u otro editor de Python y ejecutarlo desde allí.

Alcance de esta guía

Los ejercicios 1 al 10 corresponden a los ejemplos resueltos que ya venían en la actividad; se conservaron sin cambiar su lógica ni sus valores. A continuación se documentan los ejercicios 11 al 20.

Ejercicios documentados

11. Calculadora de IMC	12. Ticket de Compra de Supermercado
13. Verificador de Velocidad Permitida	14. Generador de Correo Electrónico Institucional
15. Cálculo de Propinas	16. Conversor de Edad a Días de Vida
17. Evaluación de Requisitos para Crédito	18. Formato de Tiempo en Canción/Video
19. Calculadora de Autonomía de Combustible	20. Ficha Completa de Producto

11. Calculadora de IMC

Requerimiento de la actividad: Calcula el IMC a partir del peso en kilogramos y la altura en metros.

Archivo: 11_calculadora_imc.py

```
# Calculadora de IMC
peso_kg = 67.0
altura_m = 1.75
imc = peso_kg / (altura_m ** 2)
print(f"IMC calculado: {imc:.2f}")
```

Cómo funciona: El IMC se obtiene con peso / (altura ** 2). El operador ** eleva la altura al cuadrado y :.2f muestra dos decimales.

Cómo ejecutar: python 11_calculadora_imc.py

Salida de referencia

```
IMC calculado: 21.88
```

Conceptos clave: Variables float, potencia con ** y formato decimal con :.2f.

12. Ticket de Compra de Supermercado

Requerimiento de la actividad: Define dos productos con precio y cantidad, calcula sus subtotales y el total a pagar.

Archivo: 12_ticket_supermercado.py

```
# Ticket de compra de supermercado
precio_producto1 = 3.50
cantidad_producto1 = 2
precio_producto2 = 2.50
cantidad_producto2 = 3

total_producto1 = precio_producto1 * cantidad_producto1
total_producto2 = precio_producto2 * cantidad_producto2
total_pagar = total_producto1 + total_producto2

print(f"Producto 1: ${precio_producto1:.2f} x {cantidad_producto1} = ${total_producto1:.2f}")
print(f"Producto 2: ${precio_producto2:.2f} x {cantidad_producto2} = ${total_producto2:.2f}")
print(f"TOTAL A PAGAR: ${total_pagar:.2f}")
```

Cómo funciona: Cada precio se multiplica por su cantidad para obtener un subtotal. Los subtotales se suman y se muestran mediante f-strings.

Cómo ejecutar: python 12_ticket_supermercado.py

Salida de referencia

```
Producto 1: $3.50 x 2 = $7.00
Producto 2: $2.50 x 3 = $7.50
TOTAL A PAGAR: $14.50
```

Conceptos clave: Variables, operadores aritméticos, tipos de datos, expresiones y f-strings.

13. Verificador de Velocidad Permitida

Requerimiento de la actividad: Compara la velocidad del vehículo con el límite y almacena el resultado en un booleano.

Archivo: 13_verificador_velocidad.py

```
# Verificador de velocidad permitida
velocidad_vehiculo = 67.0
limite_velocidad = 60.0
exceso_velocidad = velocidad_vehiculo &gt; limite_velocidad
print(f"Velocidad: {velocidad_vehiculo} km/h | Límite: {limite_velocidad} km/h | ¿Exceso de velocidad?: {exceso_velocidad}")
```

Cómo funciona: Se compara velocidad_vehiculo > limite_velocidad. El resultado booleano indica si se superó el límite.

Cómo ejecutar: python 13_verificador_velocidad.py

Salida de referencia

```
Velocidad: 67.0 km/h | Límite: 60.0 km/h | ¿Exceso de velocidad?: True
```

Conceptos clave: Variables, operadores, tipos de datos, expresiones y f-strings.

14. Generador de Correo Electrónico Institucional

Requerimiento de la actividad: Limpia nombre y apellido, los pasa a minúsculas y construye el correo institucional.

Archivo: 14_generador_correo_institucional.py

```
# Generador de correo electrónico institucional
nombre = " Alejandro "
apellido = " Quintero "
correo = f"{nombre.strip().lower()}.{apellido.strip().lower()}@sena.edu.co"
print(f"Correo institucional: {correo}")
```

Cómo funciona: strip() elimina espacios accidentales y lower() transforma las cadenas a minúsculas; luego se unen con el dominio institucional.

Cómo ejecutar: python 14_generador_correo_institucional.py

Salida de referencia

```
Correo institucional: alejandro.quintero@sena.edu.co
```

Conceptos clave: Cadenas, métodos strip() y lower(), variables y f-strings.

15. Cálculo de Propinas

Requerimiento de la actividad: Calcula el total de una cuenta agregando propina del 10% y del 15%.

Archivo: 15_calculo_propinas.py

```
# Cálculo de propinas
cuenta_total = 80000.0
propina_10 = cuenta_total * 0.10
propina_15 = cuenta_total * 0.15
total_10 = cuenta_total + propina_10
total_15 = cuenta_total + propina_15

print(f"Cuenta: ${cuenta_total:.2f}")
print(f"Con 10% de propina: ${total_10:.2f}")
print(f"Con 15% de propina: ${total_15:.2f}")
```

Cómo funciona: Se calculan por separado las propinas del 10% y del 15%. Cada valor se suma a la cuenta original para obtener los totales sugeridos.

Cómo ejecutar: python 15_calculo_propinas.py

Salida de referencia

```
Cuenta: $80000.00
Con 10% de propina: $88000.00
Con 15% de propina: $92000.00
```

Conceptos clave: Variables, operadores, expresiones y f-strings.

16. Conversor de Edad a Días de Vida

Requerimiento de la actividad: Convierte una edad en años a una cantidad aproximada de días usando 365 días por año.

Archivo: 16_edad_a_dias.py

```
# Conversor de edad a días de vida
edad = 17
dias_vividos = edad * 365
print(f"Edad: {edad} años | Días aproximados vividos: {dias_vividos}")
```

Cómo funciona: La edad almacenada como entero se multiplica por 365 para obtener una cantidad aproximada de días vividos.

Cómo ejecutar: python 16_edad_a_dias.py

Salida de referencia

```
Edad: 17 años | Días aproximados vividos: 6205
```

Conceptos clave: Variables enteras, multiplicación y f-strings.

17. Evaluación de Requisitos para Crédito

Requerimiento de la actividad: Aprueba el crédito sólo si los ingresos superan 2000 y no existen deudas.

Archivo: 17_requisitos_credito.py

```
# Evaluación de requisitos para crédito
ingresos_mensuales = 2500.0
tiene_deudas = False
credito_aprobado = ingresos_mensuales &gt; 2000 and not tiene_deudas
print(f"Ingresos mensuales: ${ingresos_mensuales:.2f} | ¿Tiene deudas?: {tiene_deudas} | ¿Crédito aprobado?: {credito_aprobado}")
```

Cómo funciona: La expresión combina >, and y not. El crédito se aprueba únicamente si los ingresos superan 2000 y tiene_deudas es False.

Cómo ejecutar: python 17_requisitos_credito.py

Salida de referencia

```
Ingresos mensuales: $2500.00 | ¿Tiene deudas?: False | ¿Crédito aprobado?: True
```

Conceptos clave: Operadores relacionales y lógicos: >, and y not.

18. Formato de Tiempo en Canción/Video

Requerimiento de la actividad: Convierte 215 segundos en minutos completos y segundos sobrantes.

Archivo: 18_formato_tiempo.py

```
# Formato de tiempo en canción/video
duracion_segundos = 215
minutos = duracion_segundos // 60
segundos = duracion_segundos % 60
print(f"Duración: {minutos} min {segundos} seg")
```

Cómo funciona: La división entera // 60 determina los minutos completos y % 60 obtiene los segundos restantes: 3 minutos y 35 segundos.

Cómo ejecutar: python 18_formato_tiempo.py

Salida de referencia

```
Duración: 3 min 35 seg
```

Conceptos clave: División entera (//), módulo (%) y f-strings.

19. Calculadora de Autonomía de Combustible

Requerimiento de la actividad: Calcula los kilómetros estimados a partir de litros disponibles y rendimiento km/l.

Archivo: 19_autonomia_combustible.py

```
# Calculadora de autonomía de combustible
litros_tanque = 40.0
rendimiento_km_l = 15.0
autonomia_km = litros_tanque * rendimiento_km_l
print(f"Autonomía máxima estimada: {autonomia_km:.2f} km")
```

Cómo funciona: La autonomía estimada se obtiene multiplicando los litros disponibles por el rendimiento en kilómetros por litro.

Cómo ejecutar: python 19_autonomia_combustible.py

Salida de referencia

```
Autonomía máxima estimada: 600.00 km
```

Conceptos clave: Variables float, multiplicación y formato decimal con :.2f.

20. Ficha Completa de Producto

Requerimiento de la actividad: Presenta una ficha con ID, nombre, precio, existencia en inventario y comentario.

Archivo: 20_ficha_producto.py

```
# Ficha completa de producto
id_producto = 101
nombre_producto = "Cuaderno"
precio = 12500.0
en_stock = True
comentario = "Cuaderno de práctica de 100 hojas"

print(f"""
--- FICHA DEL PRODUCTO ---
ID: {id_producto}
Nombre: {nombre_producto}
Precio: ${precio:.2f}
En stock: {en_stock}
Comentario: {comentario}
""")
```

Cómo funciona: Se usan un entero para el ID, cadenas para nombre y comentario, un float para el precio y un booleano para indicar existencia. La ficha se presenta con una f-string multilínea.

Cómo ejecutar: python 20_ficha_producto.py

Salida de referencia

```
--- FICHA DEL PRODUCTO ---
ID: 101
Nombre: Cuaderno
Precio: $12500.00
En stock: True
Comentario: Cuaderno de práctica de 100 hojas
```

Conceptos clave: Tipos int, str, float y bool; f-string multilínea.